CS 224N 2024年春季 作业4 自注意力、Transformer与预训练

注意。在规划本次作业时间时，请牢记以下几点。

- 又有数学题了！
- 本次作业需要编写的PyTorch代码总量和代码复杂度均低于作业3。不过，关于如何编写代码的指导或脚手架也更少。
- 本次作业包含一个预训练步骤，在GCP上大约需要1小时，而且你需要执行两次。我们提供了与作业4类似的Colab设置笔记本。1小时的时间线是在假设使用较旧/较慢GPU的情况下对训练时间的上限估计。在更快的GPU上，预训练大约可在30-40分钟内完成。

本次作业旨在探究 Transformer 自注意力构建模块以及预训练的影响。它通过书面问题涵盖 Transformer 与自注意力的数学性质。此外，你将通过改造现有代码库获得实际系统构建的经验。作业分为书面（数学）部分和编码部分，编码部分有独立的书面问题。以下是简要概述：

1. **数学探索：**自注意力可以轻松实现哪些类型的操作？为什么我们应该使用多头自注意力等更复杂的方法？本节将通过一些数学探究来阐明自注意力和 Transformer 网络的一些动机。**注意：**对于所有问题，在需要时你应该用数学推理来证明你的答案。

2. **扩展研究代码库：**在作业的这一部分，你将获得关于 NLP 前沿研究主题的一些经验和直觉：通过预训练教会 NLP 模型关于世界的事实，并通过微调来获取这些知识。你将训练一个 Transformer 模型来尝试回答“person [x]born在哪里？”这类简单问题——而不提供任何可供得出答案的输入文本。你会发现，模型能够通过预训练学习到一些关于人们出生地的信息，并在微调期间访问这些信息来回答这些问题。

   然后，你将更仔细地审视你所构建的系统，并推理依赖这种隐式预训练知识的含义与顾虑。

1. 注意力探索（14分）

多头自注意力是Transformer的核心建模组件。在这个问题中，我们将练习使用自注意力方程，并说明为什么多头自注意力比单头自注意力更可取。

回想一下，注意力可以被视为对查询向量 $q \in \mathbb{R}^d$、一组值向量$\{v_1, \ldots, v_n\}$ $v_i \in \mathbb{R}^d$，以及一组键向量 $\{k_1, \ldots, k_n\}$ $k_i \in \mathbb{R}^d$，进行的操作，具体如下：

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

其中 $alpha = \{\alpha_1, \ldots, \alpha_n\}$ 被称为“注意力权重”。注意，输出 $c \in \mathbb{R}^d$ 是值向量的平均值，其权重相对于 α。

(a)（3分）**注意力中的复制**。 注意力机制的一个优势是它特别容易将一个值向量“复制”到输出 c。在这个问题中，我们将解释为什么会这样。

i.（2分）分布 α 通常相对“分散”；概率质量分布在许多不同的 α_i 之间。然而，情况并非总是如此。**描述**（用一句话）在什么条件下，类别分布 α 会将几乎所有的权重放在某个 α_j 上，其中 $j \in \{1, \ldots, n\}$ (i.e. $\alpha_j \gg \sum_{i \neq j} \alpha_i$)。关于查询 q 和/或

keys $\{k_1 \ \ldots \ k_n\}$?,

ii.（1分）在你于 (i) 中给出的条件下，**描述**输出 c。

(b)（2分）**两个的平均值**。与其只关注一个向量 v_j，一个 Transformer 模型可能需要整合来自多个源向量的信息。

考虑这样一种情况：我们想要整合来自**两个**向量 v_a 和 v_b的信息，它们对应的键向量分别为 k_a 和 k_b。假设（1）所有键向量都是正交的，因此对于所有 $i \neq j$都有$k_i^\top k_j = 0$；以及（2）所有键向量的范数均为1。**请找出一个查询向量**q 的表达式，使得$c \approx \frac{1}{2}(v_a + v_b)$，并**说明你的理由**。* （回想你在第（a）部分学到的内容。）

(c)（5分）**单头注意力的缺点：**在上一部分中，我们看到了它是如何可能让单头注意力同等地关注两个值的。同样的概念可以很容易地推广到任意子集的值。在这个问题中，我们将看到为什么它不是一个实用的解决方案。

考虑一组现在被随机采样的键向量 $\{k_1, \ldots, k_n\}$， $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$，其中均值 $\mu_i \in \mathbb{R}^d$ 是已知的，但协方差 Σ_i 是未知的（除非题目中另有说明）。此外，假设均值 μ_i 都是垂直的；$\mu_i^\top \mu_j = 0$ if $i \neq j$，且为单位范数， $\|\mu_i\| = 1$。

i.（2分）假设协方差矩阵为 $\Sigma_i = \alpha I \ \forall i \in \{1, 2, \ldots, n\}$，，对于极小值 α。设计一个查询 q，用 μ_i 表示，使得如前所述， $c \approx \frac{1}{2}(v_a + v_b)$，并简要说明其为何有效。

ii.（3分）尽管单头注意力对键的小扰动具有鲁棒性，但某些类型的较大扰动可能会带来更大的问题。在某些情况下，一个键向量 k_a 的范数可能比其他键向量更大或更小，但仍指向与 μ_a.†相同的方向。举例来说，让我们考虑项目 a 的协方差为 $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ 对于趋于零的 α （如图1所示）。这导致 k_a 大致指向与 μ_a相同的方向，但幅度上存在较大方差。进一步，令 $\Sigma_i = \alpha I$ 对所有 $i \neq a$成立。

*提示：虽然softmax函数永远不会 *exactly* 对这两个向量取平均，但你可以通过在表达式中使用较大的标量倍数来接近这一效果。

†与原始Transformer不同，较新的Transformer模型在注意力之前应用层归一化。在这些pre-layernorm模型中，键的范数不会相差太大，这使得本题中的情况不太可能发生。

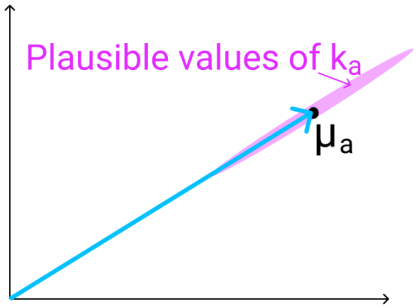


图1：向量 μ_a （此处以二维形式作为示例展示），其中 k_a 的可能取值范围以红色显示。如前所述， k_a 的方向与 μ_a大致相同，但可能具有更大或更小的模长。

当你多次采样 $\{k_1, \ldots, k_n\}$，并使用你在第i部分中定义的 q向量时，对于不同的样本，你预期向量 c 在定性上会是什么样子？思考一下它与第（i）部分有何不同，以及 c的方差会受到怎样的影响。

(d)（3分）**多头注意力的优势：**现在我们将看到多头注意力的一些强大之处。我们将考虑一个简单版本的多头注意力，它与我们之前介绍的单头自注意力完全相同，区别在于定义了两个查询向量（q_1 和 q_2），从而产生一对向量（c_1 和 c_2），每个向量都是单头注意力根据其各自查询向量得到的输出。多头注意力的最终输出是它们的平均值，$\frac{1}{2}(c_1 + c_2)$。

如同问题1(c)中一样，考虑一组随机采样的键向量 $\{k_1, \ldots, k_n\}$， $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$，其中均值 μ_i 已知，但协方差 Σ_i 未知。同样如前所述，假设均值 μ_i 相互正交； $\mu_i^\top \mu_j = 0$ if $i \neq j$，且为单位范数，$\|\mu_i\| = 1$。

i.（1分）假设协方差矩阵为 $\Sigma_i = \alpha I$，其中 α趋近于零。设计 q_1 和 q_2，以 μ_i 表示，使得 c 近似等于 $\frac{1}{2}(v_a + v_b)$。注意 q_1 和 q_2应具有不同的表达式。

ii.（2分）假设协方差矩阵为 $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$，其中$\alpha$趋近于零，而对于所有 $i \neq a$，协方差矩阵为 $\Sigma_i = \alpha I$ 。取你在第i部分设计的查询向量 q_1 和 q_2 。定性地来看，你预期输出 c 在不同的键向量样本中会呈现怎样的形态？请从 c_1 和 c_2的方差角度简要解释。你可以忽略$k_a^\top q_i < 0$的情况。

（e）（1分）基于第（d）部分，简要总结多头注意力如何克服你在第（c）部分中指出的单头注意力的缺点。

2. 位置嵌入探索（6分）

位置嵌入是Transformer架构的重要组成部分，它使模型能够根据词元在序列中的位置对其进行区分。在本问题中，我们将探讨Transformer中位置嵌入的必要性以及如何设计它们。

回顾一下，Transformer 架构的关键组成部分是自注意力层和前馈神经网络层。给定一个输入张量 $\mathbf{X} \in \mathbb{R}^{T \times d}$，其中 T 是序列长度， d 是隐藏维度，自注意力层会进行如下计算：

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$
$$\mathbf{H} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

其中 $\mathbf{W}_Q$、$\mathbf{W}_K$、$\mathbf{W}_V \in \mathbb{R}^{d\times d}$ 是权重矩阵， $\mathbf{H} \in \mathbb{R}^{T\times d}$ 是输出。

接下来，前馈层应用以下变换：

$$\mathbf{Z} = \mathrm{ReLU}(\mathbf{H}\mathbf{W}_1 + \mathbf{1}\cdot\mathbf{b}_1)\mathbf{W}_2 + \mathbf{1}\cdot\mathbf{b}_2$$

其中$\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d\times d}$和$\mathbf{b}_1, \mathbf{b}_2 \in \mathbb{R}^{1\times d}$是权重和偏置； $\mathbf{1} \in \mathbb{R}^{T\times 1}$是一个全1向量[‡]；而$\mathbf{Z} \in \mathbb{R}^{T\times d}$是最终输出。

（请注意，为简单起见，我们省略了Transformer架构的一些细节。）

(a)（4分）**置换输入**。

i.（3分）假设我们对输入序列 $\mathbf{X}$ 进行置换，使得各个词元被随机打乱。这可以表示为乘以一个置换矩阵 $\mathbf{P} \in \mathbb{R}^{T\times T}$，即 $\mathbf{X}_{\text{perm}} = \mathbf{P}\mathbf{X}$。（关于置换矩阵的回顾，请参见 Wikipedia 。）

证明 对于置换后的输入 $\mathbf{X}^{\text{perm}}$，输出 $\mathbf{Z}_{\text{perm}}$ 将为 $\mathbf{Z}^{\text{perm}} = \mathbf{P}\mathbf{Z}$。

已知对于任意置换矩阵$\mathbf{P}$ 和任意矩阵 $\mathbf{A}$，以下关系成立：

$\mathrm{softmax}(\mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}\,\mathrm{softmax}(\mathbf{A})\,\mathbf{P}^\top$ 和 $\mathrm{ReLU}(\mathbf{P}\mathbf{A}) = \mathbf{P}\,\mathrm{ReLU}(\mathbf{A})$。

ii.（1分）思考你在第i部分推导出的结果的含义。**解释**为什么Transformer模型的这一特性在处理文本时可能会带来问题。

（b）（2分）**位置嵌入**是编码序列中每个词元位置的向量。它们被加到输入词嵌入上，然后再输入到Transformer中。

一种方法是使用位置和嵌入维度的固定函数来生成位置嵌入。如果输入词嵌入为 $\mathbf{X} \in \mathbb{R}^{T\times d}$，则位置嵌入$\Phi \in \mathbb{R}^{T\times d}$ 的生成方式如下：

$$\Phi_{(t,2i)} = \sin\left(t/10000^{2i/d}\right)$$
$$\Phi_{(t,2i+1)} = \cos\left(t/10000^{2i/d}\right)$$

其中 $t \in \{0, 1, \ldots T-1\}$ 和 $i \in \{0, 1, \ldots d/2-1\}$[§]。

具体来说，位置嵌入被加到输入词嵌入上：

$$\mathbf{X}_{\text{pos}} = \mathbf{X} + \Phi$$

i.（1分）你认为位置嵌入会有助于解决你在（a）部分中发现的问题吗？如果有帮助，请解释如何有帮助；如果没有帮助，请解释为什么没有帮助。

ii.（1分）输入序列中两个不同token的位置嵌入可以相同吗？如果可以，请举一个例子。如果不可以，请解释原因。

3. 预训练Transformer模型与知识访问（35分）

你将训练一个Transformer来执行一项涉及访问世界知识的任务——这些知识并非通过任务的训练数据提供（至少在你希望泛化到训练集之外时是如此）。你会发现它在这项任务上或多或少会完全失败。然后你将学习如何

在包含世界知识的维基百科文本上预训练该Transformer，并发现在该Transformer上针对同一知识密集型任务进行微调，能使模型访问到一些在预训练阶段
学到的知识。你会发现这使模型在留出的开发集上能够显著高于随机水平地完成任务。

你拿到的代码是 Andrej Karpathy 的`minGPT` 的一个分支。它比大多数研究代码更友好，因为它相对简单且透明。`minGPT` 中的“GPT”指的是 OpenAI 的 Transformer 语言模型，最初在这篇论文 [1]中描述。

[‡]与$\mathbf{1}$ 的外积表示广播操作，并使前馈网络的记号在数学上严谨。[§]这里假设 d 为偶数，这在大多数模型中通常是如此。

与之前的作业一样，你会希望先在本地机器上开发，然后在 GCP/Colab 上运行训练。你可以使用之前作业中相同的 conda 环境进行本地开发，并使用相同的流程在 GPU 上进行训练。¶

训练大约需要 3 小时，所以请合理安排时间！我们提供了一个示例 Colab，其中包含需要 GPU 训练的命令。**请注意，数据集多进程处理在没有 GPU 的本地机器上可能会失败，因此若要在本地调试，你可能需要将`numworkers` 改为 0**。

你在此代码库中的工作如下：

(a)（0 分）**查看演示**。
在 mingpt-demo/ 文件夹中有一个 Jupyter notebook play char.ipynb，它训练并采样自一个 Transformer 语言模型。请（在本地计算机上）查看它，以大致了解它如何定义和训练模型。你在下面编写的一些代码将受到此 notebook 中所见内容的启发。

请注意，本部分无需编写任何代码、运行笔记本或提交书面答案。

(b)（0分）**阅读 src/dataset.py 中的 NameDataset，这是我们用于读取姓名-出生地配对的数据集**。

我们将使用预训练模型完成的任务是尝试获取知名人物的出生地，即其维基百科页面中所记载的信息。我们可以将其视为一种特别简单的问答形式：

问： *[person]* 出生于哪里？
答： *[place]*

从现在起，你将使用 src/ 文件夹中的内容。mingpt-demo/ 中的代码在本作业中不会被修改或评估。在 dataset.py 中，你会找到 NameDataset 类，它读取包含姓名/地点配对的 TSV（制表符分隔值）文件，并生成上述形式的样本，供我们输入 Transformer 模型。

为了了解我们将要使用的示例，如果你运行以下代码，它将在训练集出生地 train.tsv 上加载你的 NameDataset，并打印出几个示例。

```
python src/dataset.py namedata
```

请注意，你不需要为这一部分编写任何代码或提交书面答案。

(c)（0 分）**实现微调（不进行预训练）**。
看一下 run.py。它有一些骨架代码，指定了你最终需要作为命令行参数处理的标志。特别是，你可能想用这段代码来预训练、微调或评估一个模型。目前，我们将重点关注不进行预训练情况下的微调函数。

借鉴 play char.ipynb 文件中的训练代码，编写代码通过 NameDataset 类中的示例在姓名/出生地数据集上微调一个 Transformer 模型。目前，实现不进行预训练的情况（即从头创建一个模型，并在部分 (b) 的出生地预测任务上训练它）。你需要修改代码中标记为［part c］的两个部分：一个用于初始化模型，一个用于微调模型。请注意，目前你只需要在标记为“vanilla”的情况下初始化模型（稍后在部分 (g) 中，我们将探索一个模型变体）。使用 run.py 代码中为 Trainer 指定的超参数。

另外，请查看为你实现的 评估代码。它从训练好的模型中采样预测结果，并调用 evaluate places() 来获取正确地点预测的总百分比。你将在第(d)部分运行此代码来评估你训练好的模型。

¶有关 GCP 的复习内容，请参阅 CS224n GCP 指南。

这是后续部分（包括第d部分）的中间步骤，其中包含可用于检查你实现的命令。此部分无需书面回答。

提示： run.py 和 play char.ipynb 都使用 minGPT，因此这部分的代码将与 play char.ipynb 中的训练代码类似。

(d)（4分）**进行预测（不进行预训练）**。

在出生地数据 birth places_train.tsv 上训练你的模型，并在 birth dev_.tsv 上进行评估。具体来说，你现在应该能够运行以下三条命令：

```
# Train on the names dataset
python src/run.py finetune vanilla wiki.txt \
       --writing_params_path vanilla.model.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.model.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path vanilla.nopretrain.dev.predictions

# Evaluate on the test set, writing out predictions
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.model.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path vanilla.nopretrain.test.predictions
```

训练将耗时不到10分钟（在GCP上）。报告你的模型在开发集上的准确率（如上述第二条命令所打印的结果）。与作业3类似，我们在作业4中也提供了Tensorboard日志记录用于调试。可以通过 tensorboard --logdir expt/ 启动。如果准确率远低于10%，请不要惊讶；我们将在第4部分深入探讨原因。作为参考点，我们还想计算如果模型对开发集中每个人的出生地都只预测“London”所能达到的准确率。填写 london baseline.py 来计算该方法的准确率，并将结果报告在文件中。你应该能够利用现有代码，使该文件只有几行。

(e)（10分）**为预训练定义一个跨度损坏函数**。

In the file `src/dataset.py`, implement the `__getitem__()` function for the dataset class `CharCorruptionDataset`。请遵循`dataset.py`中注释所提供的说明。跨度损坏在T5论文 [2]中进行了探讨。它随机选择文档中的文本跨度，并将其替换为唯一标记（加噪）。模型接收这种加噪后的文本，并被要求输出每个唯一哨兵标记后跟该哨兵在输入中所替换的标记的模式。在本问题中，你将实现一个简化版本，仅遮蔽单个字符序列。

本题将通过自动评分器进行评分，依据是你的跨度损坏函数是否实现了我们规范的一些基本属性。我们将使用自己的数据实例化CharCorruptionDataset，并从中抽取示例。

为帮助你调试，如果你运行以下代码，它将从预训练数据集wiki.txt上的CharCorruptionDataset中采样几个示例，并为你打印出来。

```
python src/dataset.py charcorruption
```

(f)（10分）**预训练、微调并进行预测。训练预算约为1小时**。现在填写 run.py 中的预训练部分，该部分将在跨度损坏任务上预训练模型。

此外，修改你的微调部分，以处理带预训练的微调情况。具体来说，如果 bash 命令中提供了预训练模型的路径，则在出生地预测任务上微调之前加载该模型。在 wiki.txt 上预训练你的模型（大约需要 40-60 分钟），在 NameDataset 上微调并评估它。具体来说，你应该能够运行以下四个命令：

```
# Pretrain the model
python src/run.py pretrain vanilla wiki.txt \
       --writing_params_path vanilla.pretrain.params

# Finetune the model
python src/run.py finetune vanilla wiki.txt \
       --reading_params_path vanilla.pretrain.params \
       --writing_params_path vanilla.finetune.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.finetune.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path vanilla.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate vanilla wiki.txt \
       --reading_params_path vanilla.finetune.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path vanilla.pretrain.test.predictions
```

我们预计开发集准确率至少为15%，并预期在留出测试集上也能达到类似的准确率。

(g)（11分）**编写并尝试一种不同类型的位置嵌入（训练时间预算约1小时）**

在上一部分中，你使用了原始Transformer模型，它使用的是可学习的位置嵌入。在书面部分中，你还了解了原始Transformer论文中使用的正弦位置嵌入。在本部分中，你将实现一种不同类型的位置嵌入，称为*RoPE*（旋转位置嵌入）[3]。

RoPE是一种固定的位置嵌入，旨在编码相对位置而非绝对位置。绝对位置的问题在于，如果Transformer在远大于其训练时所用上下文长度（例如128）的上下文长度（例如1000）上运行，其表现将不佳，因为位置嵌入的分布会与训练时所用的分布非常不同。像RoPE这样的相对位置嵌入缓解了这一问题。

给定一个包含两个特征 $x_t^{(1)}$ 和 $x_t^{(2)}$ 的特征向量，位于序列中的位置 t，RoPE 位置嵌入定义为：

$$\text{RoPE}(x_t^{(1)}, x_t^{(2)}, t) = \begin{pmatrix} \cos t\theta & -\sin t\theta \\ \sin t\theta & \cos t\theta \end{pmatrix} \begin{pmatrix} x_t^{(1)} \\ x_t^{(2)} \end{pmatrix}$$

其中 θ 是一个固定角度。对于两个特征，RoPE 操作对应于将特征按角度 $t\theta$ 进行二维旋转。注意，该角度是位置 t 的函数。

对于 d 维特征，RoPE 应用于每一对特征，其角度 θ_i 定义为 $\theta_i = 10000^{-2(i-1)/d} \quad i \in \{1, 2, \ldots, d/2\}$，。

$$\begin{pmatrix} \cos t\theta_1 & -\sin t\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin t\theta_1 & \cos t\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos t\theta_2 & -\sin t\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin t\theta_2 & \cos t\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos t\theta_{d/2} & -\sin t\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin t\theta_{d/2} & \cos t\theta_{d/2} \end{pmatrix} \begin{pmatrix} x_t^{(1)} \\ x_t^{(2)} \\ x_t^{(3)} \\ x_t^{(4)} \\ \vdots \\ x_t^{(d-1)} \\ x_t^{(d)} \end{pmatrix} \tag{3}$$

最后，RoPE 不是将位置嵌入加到输入嵌入上，而是应用于所有 Transformer 层中注意力块内每个头的键向量和查询向量。

i. (2 p点) Using 旋转解释 p释 , RoPE 操p作可以看作是将复数 $x_t^{(1)} + ix_t^{(2)}$ 旋转角度 $t\theta$。回想一下，这对应于乘以 $e^{it\theta} = \cos t\theta + i \sin t\theta$。

对于更高维的特征向量，这种解释使我们能够更高效地计算方程 3 。具体来说，我们可以将 RoPE 操作重写为两个向量的逐元素乘法（记为 $\odot$），如下所示:

$$\begin{pmatrix} \cos t\theta_1 + i \sin t\theta_1 \\ \cos t\theta_2 + i \sin t\theta_2 \\ \vdots \\ \cos t\theta_{d/2} + i \sin t\theta_{d/2} \end{pmatrix} \odot \begin{pmatrix} x_t^{(1)} + ix_t^{(2)} \\ x_t^{(3)} + ix_t^{(4)} \\ \vdots \\ x_t^{(d-1)} + ix_t^{(d)} \end{pmatrix} \tag{4}$$

证明方程3 中的向量元素可以从方程 4中得到。注意，要使两个表达式相等，需要进行一些额外的操作，如重塑，但不需要提供详细的推导过程即可获得满分。

ii.（1 分）**相对嵌入**。现在我们将展示，两个向量在位置 t_1 和 t_2 处的 RoPE 嵌入的点积仅取决于相对位置 $t_1 - t_2$ 。

为简单起见，我们将假设二维特征向量（例如 $[a, b]$），并使用它们的复数表示（例如 $a + ib$）。

证明 $\langle \text{RoPE}(z_1, t_1), \text{RoPE}(z_2, t_2) \rangle = \langle \text{RoPE}(z_1, t_1 - t_2), \text{RoPE}(z_2, 0) \rangle$ 其中 $\langle \cdot, \cdot \rangle$ 表示点积，$\text{RoPE}(z, t)$ 是向量 z 在位置 t 处的 RoPE 嵌入。

（提示：以复数表示的向量的点积由 $\langle z_1 \ z_2 \rangle = \text{Re}(z_1 z_2)$, 给出。对于复数 $z = a + ib$ （a, $b \in \mathbb{R}$），$\text{Re}(z) = a$ 表示 z 的实部，而$\bar{z} = a - ib$ 是 z的复共轭。）

iii.（8分）在提供的代码中，RoPE 是通过 src/attention.py 中的函数 precompute rotary_emb 和 apply rotary emb 实现的。你需要实现这些函数以及 src/attention.py 和 src/run.py 中标记为 [part g] 的代码部分，以便在模型中使用 RoPE。在 span corruption 任务上训练一个使用 RoPE 的模型，并在 birthplace prediction 任务上对其进行微调。具体来说，你应该能够运行以下四个命令:

```
# Pretrain the model
python src/run.py pretrain rope wiki.txt \
       --writing_params_path rope.pretrain.params

# 微调模型
```

```
python src/run.py finetune rope wiki.txt \
       --reading_params_path rope.pretrain.params \
       --writing_params_path rope.finetune.params \
       --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate rope wiki.txt \
       --reading_params_path rope.finetune.params \
       --eval_corpus_path birth_dev.tsv \
       --outputs_path rope.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate rope wiki.txt \
       --reading_params_path rope.finetune.params \
       --eval_corpus_path birth_test_inputs.tsv \
       --outputs_path rope.pretrain.test.predictions
```

我们将根据模型在测试集上是否达到至少30%的准确率来评分，测试集的答案已保留。

4. 预训练知识的考量（5分）

请键入这些书面问题的答案（让助教们轻松一些）。

(a)（1分）简要解释为什么预训练（原始）模型能够达到10%以上的准确率，而非预训练模型则不能。

(b)（2分）查看预训练+微调原始模型的一些正确预测以及一些错误。我们认为你会发现，仅从输出上看，无法判断模型是检索到了正确的出生地，还是编造了一个错误的出生地。考虑这对涉及预训练NLP组件的面向用户系统的影响。提出两个**不同的**理由，说明这种模型行为（即无法判断是检索还是编造）为何可能对此类应用造成担忧，并为每个理由举一个例子。

(c)（2分）如果你的模型在预训练时没有见过某个人的名字，而且这个人在微调时也未被见过，那么它不可能“学会”这个人住在哪里。然而，如果被问到，你的模型仍会为该人的名字生成某个内容作为预测的出生地。请简要描述你的模型可能采取的一种策略来预测该人名字的出生地，以及这对使用此类应用所引发的一个伦理问题。

（虽然4b讨论了虚构预测可能引发的问题，4c则要求说明一种机制，
该模型可能用于生成微调时未见过的出生地的机制，以及为什么这种机制可能存在问题。）

提交说明

你需要在GradeScope上以两次提交的形式提交本次作业——一次用于**作业 4 [编码]**，另一次用于**作业 4 [书面]**：

1. 验证以下文件是否存在于您作业目录中的这些指定路径下：
   - 无预训练模型与预测：vanilla.model.params、vanilla.nopretrain.dev.predictions, vanilla.nopretrain.test.predictions
   - 伦敦基线准确率：london baseline accuracy.txt
   - 预训练-微调模型及预测结果：vanilla.finetune.params、vanilla.pretrain.dev.predictions、vanilla.pretrain.test.predictions

- RoPE 模型和预测结果：rope.finetune.params、rope.pretrain.dev.predictions、rope.pretrain.test.predictions

2. 运行 collect_submission.sh（在 Linux/Mac 上）或 collect_submission.bat（在 Windows 上）以生成你的 assignment4.zip 文件。

3. 将你的 assignment4.zip 文件上传到 GradeScope 的 **Assignment 4 [coding]**。

4. 检查公开的自动评分测试是否正确通过。

5. 将你对第1题、第2题部分内容以及第3题的书面解答上传至GradeScope的**作业4[书面]**。请正确添加标签！

参考文献

[1] Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding with unsupervised learning. *Technical report, OpenAI* (2018).

[2] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research 21*, 140 (2020), 1–67.

[3] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing 568* (2024), 127063.